



Universidad Politécnica de Querétaro

Ingeniería en Sistemas computacionales

Materia: Programación web

“Documento de investigación – HU_1.6”

Docente: José Javier Moya Moya

Por: Andrés Joshua León Barranco

Grupo: S-204

10/nov/2025

1. Objetivo

El objetivo de esta auditoría es realizar una revisión del código base existente de la plataforma Mercy Food para identificar áreas de "deuda técnica". La deuda técnica se refiere a cualquier código que, aunque funcional, es ineficiente, difícil de mantener, repetitivo o no sigue las mejores prácticas, lo que podría causar problemas en futuras expansiones.

2. Metodología

La auditoría se centró en la revisión de los Controladores (app/Http/Controllers/), las Rutas (routes/web.php) y los archivos JavaScript (public/js/).

3. Hallazgos de deuda técnica

Se han identificado cuatro áreas principales de deuda técnica que deberían abordarse en futuros Sprints para mejorar la mantenibilidad y escalabilidad de la aplicación.

Hallazgo 1: Lógica de "Pago simulado" (Deuda funcional)

- Archivo afectado: app/Http/Controllers/PaymentController.php.
- Descripción del problema: El controlador de pagos actual es una simulación. No procesa un pago real, lo que representa la mayor deuda funcional del proyecto.
- Plan de Acción: Esta deuda será saldada en el Sprint 4 con la implementación de la HU_4.1 (Backend de PayPal Sandbox).

Hallazgo 2: Controladores "Gordos" (Fat Controllers)

- Archivos afectados: app/Http/Controllers/PedidoController.php, app/Http/Controllers/PlatilloController.php.
- Descripción del problema: Los controladores están asumiendo demasiada responsabilidad (lógica de negocio) que no les corresponde.
 1. PedidoController: El método store() probablemente calcula el subtotal, el costo de envío y la comisión de la plataforma directamente dentro del controlador.
 2. PlatilloController: Los métodos store() y update() seguramente manejan la lógica de subir y guardar la imagen del platillo.
- Plan de acción: A futuro, esta lógica debería refactorizarse y moverse a "Clases de Servicio" (Service Classes), por ejemplo, PedidoService.php y FileStorageService.php, para que los controladores solo se encarguen de manejar la petición (Request) y la respuesta (Response).

Hallazgo 3: Código JavaScript monolítico

- Archivo afectado: public/js/dashboard.js.
- Descripción del problema: El archivo dashboard.js maneja la lógica AJAX para todos los perfiles (Cliente, Restaurante, Repartidor). A medida que se añadan más funciones, este archivo se volverá inmanejable.
- Plan de acción: A futuro, este archivo debería dividirse en módulos específicos (ej. cliente-pedidos.js, restaurante-menu.js) o considerar el uso de un framework de frontend ligero (como Vue.js o Alpine.js) para manejar la reactividad.

Hallazgo 4: Código duplicado (Lógica de redirección)

- Archivo afectado: app/Http/Controllers/AuthController.php.
- Descripción del problema: El método login() contiene un bloque switch (\$user->role) para redirigir al usuario a su dashboard correcto. Esta misma lógica deberá repetirse en la HU_2.1 (Login con Google).
- Plan de acción: Durante el Sprint 2, al implementar la HU_2.1, se debe crear un método privado (ej. private function redirectUserByRole(\$user)) dentro del AuthController para centralizar esta lógica y que tanto el login normal como el de Google llamen a esta única función.